

Ghid complet pentru examenul de IA - KR

Knowledge Representation / Rezolvarea problemelor prin cautare

Teorie esentiala, asocieri rapide, exemple si exercitii tip examen rezolvate

Cum se foloseste acest PDF:

- Invata mai intai sabloanele: formulare problema, cautare pe graf, euristica, CSP, minimax, retea bayesiana.
- Apoi refa exercitiile rezolvate fara sa te uiti la solutie.
- Ultimele pagini sunt o fisa rapida de memorat inainte de examen.

Realizat pentru pregatire rapida pe modelul de examen KR

Contents

1	Ce se cere in modelul de examen si cum recunosti cerinta	2
2	Agenti inteligenti si proprietatile mediilor	2
2.1	Agent inteligent	2
2.2	Proprietatile mediilor	2
2.3	Asocieri rapide	3
3	Formularea unei probleme de cautare	3
3.1	Sablonul pe care il scrii la examen	3
3.2	Exemplu: doi agenti in labirint	3
3.3	Alte sabloane de stare	4
4	Cautare neinformatata	4
4.1	Frontiera	4
4.2	Proprietati de invatat	4
5	Cautare informata: Greedy, euristici si A*	5
5.1	Functia euristica	5
5.2	Greedy Best-First Search	5
5.3	A*	5
5.4	Euristica admisibila	5
5.5	Euristica consistenta	6
5.6	Euristici uzuale	6
5.7	Dominanta euristiciilor	6
6	Exercitiu tip examen: cautare pe graf	6
7	Cautare adversariala: Minimax	8
7.1	Cand folosesti Minimax	8
7.2	Ideea algoritmului	8
7.3	Exemplu scurt	8
8	Alpha-Beta pruning	8
8.1	Ideea	8
8.2	Reguli practice	9
8.3	Exercitiu tip examen rezolvat	9
9	Expectimax	9
9.1	Cand folosesti Expectimax	9
9.2	Noduri	9
9.3	Exemplu rezolvat	9
10	Probleme de satisfacere a constrangerilor - CSP/PSC	10
10.1	Definitie	10
10.2	Exemplu: colorarea unei harti	10
10.3	Backtracking + Forward Checking	10
10.4	Cum rezolvi la examen	10
10.5	Exercitiu tip examen rezolvat	10
11	Cautare locala si Hill Climbing	11
11.1	Ideea generala	11
11.2	Avantaje si dezavantaje	11
11.3	Exemplu: 8 regine	11

12 Algoritmi genetici	11
12.1 Ideea	11
12.2 Structura	12
12.3 Componente de memorat	12
12.4 Exemplu: N-regine	12
13 Retele bayesiene simple	12
13.1 Ce este relevant pentru modelul KR	12
13.2 Reguli de baza	12
13.3 Exercițiu tip examen rezolvat	13
14 Exerciții tip examen cu soluții	13
14.1 Exercițiul 1: formulare problema cu obiecte colectate	13
14.2 Exercițiul 2: admisibilitate	14
14.3 Exercițiul 3: consistenta	14
14.4 Exercițiul 4: Expectimax	14
14.5 Exercițiul 5: CSP mic	14
14.6 Exercițiul 6: Minimax	15
15 Asocieri rapide pentru examen	15
16 Fisa finala de memorat	15

1 Ce se cere in modelul de examen si cum recunosti cerinta

In modelul primit la KR apar cinci tipuri mari de probleme. Scopul tau este sa recunosti rapid tipul problemei si sa aplici sablonul potrivit.

Cerinta din examen	Ce teorie folosesti	Ce scrii in rezolvare
Formularea unei probleme cu agenti intr-un labirint	spatiu de stari, stare initiala, stare scop, actiuni, cost, functie succesor, factor de ramificare, numar de stari, euristica admisibila	definesti starea ca pozitia agentilor, actiunile ca produs cartezian, numeri celulele libere, alegi o euristica optimista
DFS/BFS/Greedy/A* pe graf	cautare neinformatata si informata, frontiera, prioritati	scrii frontiera pas cu pas si solutia finala; pentru Greedy folosesti h , pentru A* folosesti $g + h$
CSP/colorare harta	variabile, domenii, constrangeri, backtracking, forward checking	desenezi/descrii graful de constrangeri, alegi ordinea variabilelor, elimini culori din domeniile vecinilor
Minimax si Alpha-Beta	jocuri adversariale, MAX/MIN, retezare	calculezi valorile de jos in sus; marchezi ramurile taiate cand $\alpha \geq \beta$
Rețele bayesiene simple	distributie comuna, probabilitati conditionate, inversare arc	calculezi $P(G, T) = P(G)P(T G)$, apoi $P(G T) = P(G, T)/P(T)$

Important pentru examen: Daca nu ai timp, prioritatea este: (1) formulare problema, (2) cautari pe graf, (3) CSP, (4) Minimax/Alpha-Beta, (5) rețele bayesiene.

2 Agenti inteligenti si proprietatile mediilor

2.1 Agent inteligent

Un **agent** este o entitate care:

- percepe mediul prin senzori;
- actioneaza asupra mediului prin actuatori;
- alege actiuni pentru a atinge un scop.

Exemple: robot, masina autonoma, program care detecteaza spam, agent intr-un joc.

2.2 Proprietatile mediilor

Proprietate	Intrebare	Exemple
Complet observabil / partial observabil	Agentul vede toata starea relevanta?	sah: complet; poker: partial
Determinist / stocastic	Starea urmatoare e determinata sigur de actiune?	8-puzzle: determinist; table: stocastic
Discret / continuu	Starile si actiunile sunt finite?	8-puzzle: discret; condus masina: continuu
Episodic / secvential	Decizia curenta afecteaza deciziile viitoare?	OCR independent: episodic; sah: secvential
Static / dinamic	Mediul se schimba cat timp agentul gandeste?	sah: static; Pac-Man: dinamic

Agent / multiagent	Exista un singur agent sau mai multi?	8-puzzle: agent; sah: multiagent competitiv
--------------------	---------------------------------------	---

2.3 Asocieri rapide

- **Sah/dame:** complet observabil, determinist, discret, secvential, static, multiagent competitiv.
- **Poker:** partial observabil, stocastic, discret, secvential, static, multiagent competitiv.
- **Masina autonoma:** partial observabil, stocastic, continuu, secvential, dinamic, multiagent.
- **Pac-Man:** complet observabil de obicei, stocastic daca fantomele au comportament imprevizibil, discret, secvential, dinamic, multiagent competitiv.

3 Formularea unei probleme de cautare

3.1 Sablonul pe care il scrii la examen

Pentru orice problema de cautare, scriu:

1. **Spatiul starilor:** cum codific o stare s .
2. **Starea initiala:** $s_0 = \dots$
3. **Stari finale/scop:** conditia care defineste scopul.
4. **Actiuni:** ce poate face agentul dintr-o stare.
5. **Functia succesor:** ce stare se obtine dupa o actiune.
6. **Cost:** de obicei 1 pe mutare, daca nu se precizeaza altfel.
7. **Factor de ramificare:** numarul maxim de actiuni posibile dintr-o stare.
8. **Numar de stari:** produsul posibilitatilor componentelor starii.
9. **Euristica admisibila:** o estimare care nu supraestimeaza costul real.

3.2 Exemplu: doi agenti in labirint

Avem doi agenti M si U intr-un labirint 6×14 . Scopul este sa ajunga pe aceeasi celula.

Rezolvare. O stare retine pozitiile ambilor agenti:

$$s = ((x_M, y_M), (x_U, y_U)).$$

Daca M incepe in stanga-jos si U in dreapta-sus, putem avea:

$$s_0 = ((1, 6), (14, 1)).$$

O stare finala este orice stare in care cei doi sunt pe aceeasi celula:

$$s_f = ((x, y), (x, y)).$$

Actiunile fiecarui agent sunt:

$$\{stanga, dreapta, sus, jos, stat\}.$$

Cum agentii se misca simultan, actiunea totala este un produs cartezian:

$$A = A_M \times A_U.$$

Deci, ignorand obstacolele si marginile, factorul maxim de ramificare este:

$$b_{max} = 5 \cdot 5 = 25.$$

Daca labirintul are 84 celule si 25 obstacole, atunci sunt 59 celule libere. Cei doi agenti pot sta in aceeasi celula, deci:

$$|S| = 59^2 = 3481.$$

O euristica admisibila este jumatate din distanta Manhattan dintre cei doi:

$$h(s) = \left\lceil \frac{|x_M - x_U| + |y_M - y_U|}{2} \right\rceil.$$

Este admisibila deoarece ambii agenti se pot apropia simultan; distanta Manhattan dintre ei poate scadea cu cel mult 2 la o mutare, deci numarul real de pasi nu poate fi mai mic decat jumatate din distanta.

3.3 Alte sabloane de stare

Problema	Stare posibila	Observatie
8-puzzle	configuratia tablei 3×3 , de exemplu $[2, X, 3, 1, 8, 4, 7, 6, 5]$	X este spatiul liber
Blocuri	lista ordonata de stive $s = [S_1, S_2, \dots, S_n]$	ultimul element din stiva este varful
Labirint cu obiecte de colectat	$(pozitie, obiecte_colectate)$	adaugi energie/scor daca problema o cere
Joc X si 0	configuratia tablei + jucatorul la mutare	pentru Minimax trebuie sa stii cine muta
CSP/colorare	asignari partiale, de exemplu $A = rosu, B = ?$	fiecare pas atribuie o variabila

4 Cautare neinformatata

4.1 Frontiera

Frontiera este multimea nodurilor generate, dar inca neexpandate. Diferenta dintre algoritmi este modul in care aleg nodul urmator din frontiera.

Algoritm	Frontiera	Alege	Observatie examen
BFS	coada FIFO	cel mai vechi nod / nivel minim	optim daca toate costurile sunt egale
DFS	stiva LIFO	cel mai recent nod / adancime maxima	nu garanteaza optimul
DLS	stiva cu limita	DFS pana la adancimea l	poate rata solutia daca limita e prea mica
IDS	DFS repetat cu limite crescatoare	simuleaza BFS cu memorie de DFS	complet si optim pentru costuri egale
UCS	coada de prioritati	cost $g(n)$ minim	optim pentru costuri pozitive

4.2 Proprietati de invatat

Algoritm	Complet?	Optim?	Timp	Spatiu
----------	----------	--------	------	--------

BFS	da, daca solutia are adancime finita	da, daca toate muchiile au acelasi cost	$O(b^s)$	$O(b^s)$
DFS	nu in spatii infinite; da cu adancime finita si fara cicluri	nu	$O(b^m)$	$O(bm)$
DLS	nu daca limita e prea mica	nu	$O(b^l)$	$O(bl)$
IDS	da	da, daca toate costurile sunt egale	$O(b^s)$	$O(bs)$
UCS	da pentru costuri pozitive	da	poate fi mare	poate fi mare

Unde b = factor de ramificare, s = adancimea celei mai apropiate solutii, m = adancimea maxima, l = limita.

5 Cautare informata: Greedy, euristici si A^*

5.1 Functia euristica

O euristica $h(n)$ estimeaza costul minim ramas de la nodul n pana la scop. Proprietati de baza:

- $h(n) \geq 0$;
- $h(scop) = 0$;
- este specifica problemei;
- exemple: distanta Manhattan, distanta euclidiană, număr de piese puse gresit.

5.2 Greedy Best-First Search

Greedy alege nodul cu euristica minima:

$$f(n) = h(n).$$

Este rapid, dar nu garanteaza solutia optima, pentru ca ignora costul parcurs deja.

5.3 A^*

A^* alege nodul cu:

$$f(n) = g(n) + h(n),$$

unde:

- $g(n)$ = costul de la start pana la n ;
- $h(n)$ = estimarea de la n la scop;
- $f(n)$ = estimarea costului total al unui drum prin n .

5.4 Euristica admisibila

O euristica este **admisibila** daca nu supraestimeaza niciodata costul real pana la scop:

$$h(n) \leq h^*(n), \quad \forall n,$$

unde $h^*(n)$ este costul real minim de la n la scop.

Important pentru examen: La examen, ca sa arati ca o euristica NU e admisibila, este suficient sa gasesti un singur nod n cu $h(n) > h^*(n)$.

5.5 Euristica consistenta

O euristica este **consistenta** daca pentru orice muchie (a, b) :

$$|h(a) - h(b)| \leq c(a, b).$$

Echivalent, pentru un drum orientat:

$$h(a) \leq c(a, b) + h(b).$$

Daca h este consistenta, atunci valorile $f = g + h$ nu scad de-a lungul unui drum.

Varianta A*	Conditie pentru optimalitate
A* Tree Search	h admisibila
A* Graph Search	h consistenta

5.6 Euristici uzuale

Euristica	Formula	Cand e potrivita
Manhattan	$ x - x_g + y - y_g $	grid cu miscari N, S, E, V
Euclidiana	$\sqrt{(x - x_g)^2 + (y - y_g)^2}$	spatiu continuu sau miscari libere
Chebyshev	$\max(dx , dy)$	grid cu miscari in 8 directii, cost unitar
Octile	$\max(dx, dy) + (\sqrt{2} - 1) \min(dx, dy)$	grid cu diagonale de cost $\sqrt{2}$
8-puzzle #tiles	numarul de piese gresite	euristica simpla, admisibila
8-puzzle Manhattan	suma distantelor Manhattan ale pieselor	mai informativa decat #tiles

5.7 Dominanta euristicilor

Daca doua euristici h_1, h_2 sunt admisibile si:

$$h_2(n) \geq h_1(n), \quad \forall n,$$

atunci h_2 domina h_1 si A* cu h_2 va expanda cel mult la fel de multe noduri.

Maximumul a doua euristici admisibile ramane admisibil:

$$h(n) = \max(h_1(n), h_2(n)).$$

6 Exerciitiu tip examen: cautare pe graf

Consideram graful:

$$S \rightarrow A(1), \quad S \rightarrow G(12), \quad A \rightarrow B(3), \quad A \rightarrow C(1), \quad B \rightarrow D(3), \quad C \rightarrow D(1), \quad C \rightarrow G(2), \quad D \rightarrow G(3).$$

Euristica:

$$h(S) = 3, \quad h(A) = 2, \quad h(B) = 1, \quad h(C) = 4, \quad h(D) = 2, \quad h(G) = 0.$$

a) DFS

Rezolvare. Presupunem ordine lexicografică.

$$\begin{aligned}
 &S \\
 &S - A, S - G \\
 &S - A - B, S - A - C, S - G \\
 &S - A - B - D, S - A - C, S - G \\
 &S - A - B - D - G, S - A - C, S - G.
 \end{aligned}$$

Soluție DFS:

$$S - A - B - D - G.$$

b) BFS

Rezolvare. BFS merge pe niveluri:

$$\begin{aligned}
 &S \\
 &S - A, S - G.
 \end{aligned}$$

Cum G este generat la adâncime 1, soluția este:

$$S - G.$$

c) Este h admisibilă?

Rezolvare. Verificăm dacă există un nod unde $h(n) > h^*(n)$. Din C până la G există muchia $C \rightarrow G$ de cost 2, deci:

$$h^*(C) = 2.$$

Dar:

$$h(C) = 4 > 2.$$

Deci euristica **nu este admisibilă**.

d) Greedy

Rezolvare. Greedy folosește doar h :

$$S(3)$$

Succesorii sunt:

$$S - A(2), \quad S - G(0).$$

Alege $S - G$, deoarece $h(G) = 0$.

e) A*

Rezolvare. A* folosește $f = g + h$.

$$S : 0 + 3 = 3.$$

După expandarea lui S :

$$S - A : 1 + 2 = 3, \quad S - G : 12 + 0 = 12.$$

Expandăm $S - A$:

$$S - A - B : 4 + 1 = 5, \quad S - A - C : 2 + 4 = 6, \quad S - G : 12.$$

Expandam $S - A - B$:

$$S - A - B - D : 7 + 2 = 9, \quad S - A - C : 6, \quad S - G : 12.$$

Expandam $S - A - C$:

$$S - A - C - D : 3 + 2 = 5, \quad S - A - C - G : 4 + 0 = 4, \quad S - A - B - D : 9, \quad S - G : 12.$$

Urmatorul nod minim este $S - A - C - G$, deci solutia este:

$$S - A - C - G, cost = 4.$$

7 Cautare adversariala: Minimax

7.1 Cand folosesti Minimax

Folosesti Minimax in jocuri:

- cu doi jucatori;
- de suma zero;
- deterministe;
- complet observabile;
- cu mutari alternative.

Exemple: sah, dame, X si 0.

7.2 Ideea algoritmului

- MAX vrea sa maximizeze scorul.
- MIN vrea sa minimizeze scorul lui MAX.
- La frunze avem utilitati.
- La un nod MAX luam maximul valorilor fiilor.
- La un nod MIN luam minimul valorilor fiilor.

$$V(n) = \begin{cases} U(n), & n \text{ terminal,} \\ \max_{s \in Succ(n)} V(s), & n \text{ este MAX,} \\ \min_{s \in Succ(n)} V(s), & n \text{ este MIN.} \end{cases}$$

7.3 Exemplu scurt

Radacina este MAX. Are doi fii MIN:

- primul MIN are frunze 3, 12, 8, deci valoarea lui este $\min(3, 12, 8) = 3$;
- al doilea MIN are frunze 2, 14, 5, deci valoarea lui este $\min(2, 14, 5) = 2$.

Radacina MAX alege:

$$\max(3, 2) = 3.$$

8 Alpha-Beta pruning

8.1 Ideea

Alpha-Beta calculeaza aceeasi valoare ca Minimax, dar nu mai viziteaza ramurile care nu pot influenta decizia finala.

- α = cea mai buna valoare garantata pentru MAX pana acum.
- β = cea mai buna valoare garantata pentru MIN pana acum.
- Pornim cu $\alpha = -\infty$, $\beta = +\infty$.
- Taiem cand $\alpha \geq \beta$.

8.2 Reguli practice

Nod	Actualizare	Taiere
MAX	$v = \max(v, \text{valoare_fiu})$, $\alpha = \max(\alpha, v)$	daca $v \geq \beta$, opresti restul fiilor
MIN	$v = \min(v, \text{valoare_fiu})$, $\beta = \min(\beta, v)$	daca $v \leq \alpha$, opresti restul fiilor

Important pentru examen: Formula de memorat: tai cand $\alpha \geq \beta$. Daca esti in MAX verifici $v \geq \beta$. Daca esti in MIN verifici $v \leq \alpha$.

8.3 Exercițiu tip examen rezolvat

Ai un arbore vizitat de la stanga la dreapta. Dupa calculul Minimax rezulta ca ramurile care nu mai pot influenta radacina sunt h si f .

Rezolvare. Justificarea se face asa:

1. Calculezi mai intai valorile subarborelui din stanga.
2. Radacina sau nodurile superioare obtin o limita α sau β .
3. Cand intr-un nod MIN gasesti deja o valoare $\leq \alpha$, restul fiilor nu mai pot ajuta MAX, deci se taie.
4. Cand intr-un nod MAX gasesti deja o valoare $\geq \beta$, restul fiilor nu mai pot fi acceptati de MIN, deci se taie.

9 Expectimax

9.1 Cand folosesti Expectimax

Folosesti Expectimax cand nu ai un adversar perfect rational de tip MIN, ci exista incertitudine:

- zaruri;
- aparitie aleatoare a unei piese;
- actiuni care pot esua;
- adversar cu comportament probabilistic.

9.2 Noduri

- Nod MAX: alege maximul, ca in Minimax.
- Nod sansa/EXP: calculeaza media ponderata a fiilor.

$$V_{EXP}(n) = \sum_i p_i V(s_i).$$

Daca nu se dau probabilitati, se poate folosi media aritmetica.

9.3 Exemplu rezolvat

Un nod de sansa are trei fii cu valori 8, 24, -12 si probabilitati $1/2, 1/3, 1/6$.

Rezolvare.

$$V = \frac{1}{2} \cdot 8 + \frac{1}{3} \cdot 24 + \frac{1}{6} \cdot (-12) = 4 + 8 - 2 = 10.$$

Important pentru examen: Alpha-Beta nu se aplica direct la Expectimax, deoarece un fiu nevizitat cu valoare foarte mare poate schimba media.

10 Probleme de satisfacere a constrangerilor - CSP/PSC

10.1 Definitie

O problema de satisfacere a constrangerilor are trei componente:

$$CSP = (X, D, C),$$

unde:

- $X = \{X_1, X_2, \dots, X_n\}$ este multimea variabilelor;
- $D = \{D_1, D_2, \dots, D_n\}$ sunt domeniile variabilelor;
- C este multimea constrangerilor.

O solutie este o asignare completa a variabilelor care respecta toate constrangerile.

10.2 Exemplu: colorarea unei harti

- Variabile: regiunile hartii, de exemplu A, B, C, D, E, F .
- Domenii: $D_i = \{rosu, verde, albastru\}$.
- Constrangeri: doua regiuni vecine trebuie sa aiba culori diferite.

10.3 Backtracking + Forward Checking

Backtracking atribuie valori variabilelor pe rand. Daca apare conflict, se intoarce.

Forward checking inseamna ca dupa ce alegem o valoare pentru o variabila, eliminam din domeniile vecinilor valorile care ar produce conflict.

10.4 Cum rezolvi la examen

Pasi CSP la examen:

1. Scrii variabilele.
2. Scrii domeniul fiecarei variabile.
3. Scrii constrangerile sau graful de constrangeri.
4. Alegi ordinea variabilelor. Daca se cere dupa grad: variabilele cu cei mai multi vecini primele; la egalitate lexicografic.
5. Alegi prima valoare pentru prima variabila.
6. Prin forward checking, stergi acea valoare din domeniile vecinilor.
7. Continui pana ai solutie sau un domeniu devine vid.

10.5 Exerciitiu tip examen rezolvat

Variabilele sunt A, B, C, D, E, F , domeniul este $\{rosu, verde, albastru\}$. Ordinea dupa numarul de constrangeri este:

$$B - D - F - A - C - E.$$

Rezolvare. Initial:

$$D(A) = D(B) = D(C) = D(D) = D(E) = D(F) = \{R, V, A\},$$

unde $R = rosu, V = verde, A = albastru$.

Alegem:

$$B = R.$$

Atunci eliminam R din domeniile vecinilor lui B :

$$D(D) = D(F) = D(A) = D(C) = \{V, A\}.$$

Alegem:

$$D = V.$$

Eliminam V din domeniile vecinilor lui D :

$$D(F) = \{A\}, \quad D(C) = \{A\}, \quad D(E) = \{R, A\}.$$

Alegem:

$$F = A.$$

Eliminam A din domeniile vecinilor lui F :

$$A = V, \quad E = R.$$

O solutie este:

$$B = R, D = V, F = A, A = V, C = A, E = R.$$

In total sunt 6 solutii, deoarece alegem doua culori diferite pentru B si D in $3 \cdot 2 = 6$ moduri, iar apoi restul sunt fortate.

11 Cautare locala si Hill Climbing

11.1 Ideea generala

Cautarea locala nu construieste un drum complet de la start la scop, ci pastreaza o **stare curenta** si o imbunatateste local.

- Porneste dintr-o stare oarecare, deseori aleatoare.
- Genereaza vecinii starii curente.
- Alege vecinul cel mai bun.
- Se opreste daca nu exista vecin mai bun.

11.2 Avantaje si dezavantaje

Avantaje	Dezavantaje
foarte rapida	incompleta
memorie mica	suboptimala
se aplica pe spatii foarte mari	se poate bloca in maxime/minime locale sau platouri

11.3 Exemplu: 8 regine

O stare poate fi o asignare cu o regina pe fiecare coloana. Succesorii se obtin mutand o regina pe alta linie in aceeasi coloana. Functia cost poate fi numarul perechilor de regine care se ataca; vrem minim 0.

12 Algoritmi genetici

12.1 Ideea

Algoritmii genetici mentin o populatie de solutii candidat. Fiecare individ are un fitness. Indivizii mai buni sunt selectati mai des, apoi se aplica incrucisare si mutatie.

12.2 Structura

1. Initializare populatie aleatoare.
2. Evaluare populatie prin fitness.
3. Selectie parinti.
4. Generare urmasi prin crossover.
5. Mutatie.
6. Evaluare urmasi.
7. Selectie supravietuitori.
8. Repetare pana la conditia de oprire.

12.3 Componente de memorat

Componenta	Rol
Cromozom/codificare	cum reprezentam o solutie: sir binar, permutare, vector real
Fitness	masoara calitatea solutiei
Selectie	alege indivizii mai buni pentru reproducere
Crossover	combina doi parinti, explorare
Mutatie	introduce variatie, evita blocarea in optime locale

12.4 Exemplu: N-regine

Pentru 8 regine, o codificare poate fi o permutare:

$$(s_1, s_2, \dots, s_8),$$

unde s_i este linia reginei din coloana i . Fitnessul poate fi numarul de perechi de regine care nu se ataca. Maximul este:

$$\binom{8}{2} = 28.$$

Daca exista 5 perechi care se ataca, fitnessul este:

$$28 - 5 = 23.$$

13 Retele bayesiene simple

13.1 Ce este relevant pentru modelul KR

Pentru examenul tau apare o retea bayesiana mica, cu doua variabile. Trebuie sa stii sa construisti distributia comuna si sa inversezi dependentia.

13.2 Reguli de baza

Daca avem $G \rightarrow T$, atunci:

$$P(G, T) = P(G)P(T | G).$$

Daca vrem retea inversa $T \rightarrow G$, avem nevoie de:

$$P(T) = \sum_G P(G, T)$$

si:

$$P(G | T) = \frac{P(G, T)}{P(T)}.$$

13.3 Exercițiu tip examen rezolvat

Se dau:

$$P(G = 1) = 0.1, \quad P(T = 1 \mid G = 1) = 0.8, \quad P(T = 1 \mid G = 0) = 0.3.$$

Rezolvare. Completăm:

$$P(G = 0) = 0.9, \quad P(T = 0 \mid G = 1) = 0.2, \quad P(T = 0 \mid G = 0) = 0.7.$$

Calculăm distribuția comună:

$$P(G = 1, T = 1) = 0.1 \cdot 0.8 = 0.08,$$

$$P(G = 1, T = 0) = 0.1 \cdot 0.2 = 0.02,$$

$$P(G = 0, T = 1) = 0.9 \cdot 0.3 = 0.27,$$

$$P(G = 0, T = 0) = 0.9 \cdot 0.7 = 0.63.$$

Verificare:

$$0.08 + 0.02 + 0.27 + 0.63 = 1.$$

Pentru rețeaua inversă $T \rightarrow G$:

$$P(T = 0) = 0.02 + 0.63 = 0.65,$$

$$P(T = 1) = 0.08 + 0.27 = 0.35.$$

Apoi:

$$P(G = 0 \mid T = 0) = \frac{0.63}{0.65} = \frac{63}{65},$$

$$P(G = 1 \mid T = 0) = \frac{0.02}{0.65} = \frac{2}{65},$$

$$P(G = 0 \mid T = 1) = \frac{0.27}{0.35} = \frac{27}{35},$$

$$P(G = 1 \mid T = 1) = \frac{0.08}{0.35} = \frac{8}{35}.$$

14 Exerciții tip examen cu soluții

14.1 Exercițiul 1: formulare problema cu obiecte colectate

Un robot se afla într-un grid $m \times n$, există obstacole și k obiecte de colectat. Robotul poate merge sus, jos, stanga, dreapta. Scopul este să colecteze toate obiectele și să ajungă la ieșire.

Cerinte: definește starea, starea inițială, starea finală, acțiunile, factorul de ramificare, o euristică admisibilă.

Rezolvare. Starea trebuie să țină poziția robotului și ce obiecte au fost colectate:

$$s = ((x, y), C),$$

unde $C \subseteq \{1, 2, \dots, k\}$ este mulțimea obiectelor colectate.

Starea inițială:

$$s_0 = ((x_0, y_0), \emptyset).$$

Stare finală:

$$((x_{exit}, y_{exit}), \{1, 2, \dots, k\}).$$

Acțiuni:

$$\{sus, jos, stanga, dreapta\},$$

valide dacă nu ies din grid și nu intra în obstacol.

Factor de ramificare:

$$b_{max} = 4.$$

Euristica admisibila: distanta Manhattan de la pozitia curenta pana la cel mai apropiat obiect necolectat plus o estimare optimista pana la iesire. O varianta simpla, sigur admisibila, este:

$$h(s) = \min_{o \notin C} d_{Manhattan}((x, y), o),$$

cu $h = d_{Manhattan}((x, y), exit)$ daca toate obiectele sunt colectate. Este admisibila deoarece ignora obstacolele si ignora o parte din costurile necesare.

14.2 Exerciitiul 2: admisibilitate

Se da un graf si o euristica. Pentru nodul X , costul real minim pana la scop este 5, iar $h(X) = 7$. Este euristica admisibila?

Rezolvare. Nu. Pentru admisibilitate trebuie:

$$h(X) \leq h^*(X).$$

Dar:

$$7 > 5.$$

Deci euristica supraestimeaza costul real si nu este admisibila.

14.3 Exerciitiul 3: consistenta

Avem o muchie $A \rightarrow B$ de cost 3, $h(A) = 7$, $h(B) = 2$. Este respectata consistenta pe aceasta muchie?

Rezolvare. Verificam:

$$|h(A) - h(B)| \leq c(A, B).$$

$$|7 - 2| = 5 > 3.$$

Nu este consistenta.

14.4 Exerciitiul 4: Expectimax

Un nod MAX are doi fii. Primul are valoare 6. Al doilea este nod sansa cu rezultate 10 si -2 , fiecare cu probabilitate $1/2$. Ce alege MAX?

Rezolvare. Valoarea nodului sansa este:

$$\frac{1}{2} \cdot 10 + \frac{1}{2} \cdot (-2) = 5 - 1 = 4.$$

MAX compara 6 si 4, deci alege primul fiu, cu valoare 6.

14.5 Exerciitiul 5: CSP mic

Avem trei variabile A, B, C , domeniul $\{R, V\}$ si constrangerile $A \neq B$, $B \neq C$. Gaseste toate solutiile.

Rezolvare. Daca $B = R$, atunci $A = V$ si $C = V$, deci solutia este:

$$(A = V, B = R, C = V).$$

Daca $B = V$, atunci $A = R$ si $C = R$, deci:

$$(A = R, B = V, C = R).$$

Sunt doua solutii.

14.6 Exercițiul 6: Minimax

Radacina este MAX. Are doi fii MIN. Primul MIN are frunze 4, 9, al doilea MIN are frunze 5, 2. Care este valoarea radacinii?

Rezolvare. Primul MIN:

$$\min(4, 9) = 4.$$

Al doilea MIN:

$$\min(5, 2) = 2.$$

Radacina MAX:

$$\max(4, 2) = 4.$$

Deci MAX alege primul fiu.

15 Asocieri rapide pentru examen

Daca vezi in cerinta...	Te gandesti imediat la...
„codificati o stare”	ce informatie minima trebuie retinuta pentru a continua problema
„doi agenti simultan”	starea retine pozitiile amandurora, iar actiunile se inmultesc
„numar minim de mutari”	cost unitar, BFS daca nu exista costuri diferite, A* daca ai euristica
„factor maxim de ramificare”	numar maxim de actiuni valide ignorand restrictii locale
„numar de stari”	produs de posibilitati; ai grija daca agentii pot fi pe aceeasi celula
„euristica admisibila”	nu supraestimeaza; ignora obstacole/restrictii ca sa fie optimista
„DFS/BFS/Greedy/A*”	scrii frontiera pas cu pas
„h admisibila?”	compari cu h^* ; un contraexemplu ajunge
„Graph Search”	exista CLOSED; pentru A* optimal ai nevoie de consistenta
„MAX/MIN”	calculezi de jos in sus: MAX=max, MIN=min
„Alpha-Beta”	$\alpha = -\infty$, $\beta = +\infty$, tai cand $\alpha \geq \beta$
„nod de sansa/probabilitati”	Expectimax, medie ponderata
„variabile, domenii, constrangeri”	CSP/PSC
„forward checking”	dupa asignare elimini valori din domeniile vecinilor
„distributie comuna”	inmultesti probabilitatea parintelui cu probabilitatea conditionata

16 Fisa finala de memorat

Formulare problema

- Stare, stare initiala, stare scop, actiuni, succesor, cost.
- Solutie = drum de la start la scop.
- Solutie optima = drum cu cost minim.
- b_{max} = numar maxim de succesor.

Cautari

- BFS: FIFO, pe niveluri, optim daca toate costurile sunt egale.
- DFS: LIFO, merge adanc, nu e optim.
- UCS: prioritate dupa $g(n)$, optim pentru costuri pozitive.
- Greedy: $f(n) = h(n)$, rapid, nu optim.
- A*: $f(n) = g(n) + h(n)$.

Euristici

- Admisibila: $h(n) \leq h^*(n)$.
- Consistenta: $|h(a) - h(b)| \leq c(a, b)$.
- Tree A* optim cu h admisibila.
- Graph A* optim cu h consistenta.
- $h = 0$ transforma A* in UCS.
- $h = h^*$ este perfecta, dar inutila practic pentru ca rezolva deja problema.

Minimax si Alpha-Beta

- MAX ia maxim.
- MIN ia minim.
- α = garantie pentru MAX.
- β = garantie pentru MIN.
- Tai cand $\alpha \geq \beta$.

Expectimax

- MAX ia maxim.
- EXP ia media ponderata.
- $V = \sum_i p_i V_i$.

CSP

- X = variabile.
- D = domenii.
- C = constrangeri.
- Forward checking = sterg valori imposibile din domeniile vecinilor.

Retele bayesiene

- Pentru $G \rightarrow T$: $P(G, T) = P(G)P(T | G)$.
- $P(T) = \sum_G P(G, T)$.
- $P(G | T) = P(G, T)/P(T)$.

Genetici

- Populatie, cromozom, fitness, selectie, crossover, mutatie.
- Crossover + mutatie = explorare.
- Selectia = exploatare.

Surse folosite

Material sintetizat din cursurile si modelele incarcate: Cursurile 1-7 de IA - partea de Rezolvarea problemelor si cautare, seminarul de cautare, modelul de examen KR si rezolvarea modelului.